CIS 4060 Mobile Application Development Project

Build a Mobile App of Your Own

This document covers everything about the project. Use it as your reference all term.

Note. For those that are in multiple courses, please note that there may be slightly different requirements per course, including the schedule of due dates. Please read carefully and ask if you are unsure about something.

Contents

Foreword

1. Your Mission
2. This Does Not Have to End This Semester
3. How This Works
4. Project Guidelines
5. Mobile App Categories and Ideas
6. What Goes Into a Mobile App: The Capability Map
7. Schedule
8. What Progress Looks Like
9. The Progress Video
10. The Ten Specifications
11. How Your Project Grade Is Determined
12. Tokens
13. Final Submission and Presentation
14. AI and Outside Resources

Foreword

This project exists because students asked for it.

A couple of semesters ago, students told us in class, end-of-semester surveys, and graduation exit interviews that they wanted to build something real, something of their own, using full-stack development. Enough of them said it, clearly enough, that I made room for it. I cut what was most often identified as “busy work” so this project could exist. You are getting the benefit of that, and I want you to enjoy it.

That is the spirit of the whole thing. It is yours: you choose it, you build it at your own pace, and nobody is grading how impressive it turns out. It does not have to work, does not have to be finished, and does not have to be original. What it does need is for you to keep at it and keep me in the loop. I am your manager on this, which mostly means I am the person you come to when something goes sideways, and something will for almost everyone. That is normal and expected, and it is fine as long as I hear about it. If you ever feel stressed about this project, come see me. Usually it just means the scope needs a small adjustment, and that is an easy fix early.

Some of you know people who have already built a project like this in one of my courses. Ask them what it was like. And when you are through, tell people about your project and the

experience.

1. Your Mission

Build a mobile app you actually want to build. Maybe you already have an idea you have been meaning to try. Maybe you have seen an app and wanted to make your own version. Both are good places to start, and imitation is a legitimate way to learn: rebuilding something you admire will teach you more than inventing something you do not care about.

The one firm requirement is that it must be a mobile application. Beyond that, the subject is yours. It should draw on the material we cover, and you must keep it moving across the semester. It does not have to be an original idea, and it does not have to be finished by the end of the term; projects seldom are. This project is about what you build, what you learn, and what you can show for it.

Build with Android Studio, using whatever it gives you to work with. Kotlin is the modern default and what our book uses; Java also works. Use the tools the environment hands you.

2. This Does Not Have to End This Semester

Pick something you actually care about, because you may not be done with it.

If you take another course with me, you can carry this project forward. You start from where you left off, build the next layer with the new course's material, and get credit for it there. The app you build here could grow a networked, secured back end in CIS 3970, Secure Programming Principles, where we work with threading, sockets, and security. It could get serious data structures under the hood in CIS 4020. These courses do not have to be taken in any particular order, so wherever you are in the sequence, a project you care about can travel with you.

Over two or three semesters, that adds up to something substantially more interesting than a handful of assignments, and it is something you can show people and include on your resume.

3. How This Works

You are the developer. I am your manager. The project is self-directed and you set its direction, but a working relationship has obligations: you keep your manager informed, you deliver on agreed dates, and you show real work rather than descriptions of work you intend to do.

Your project grade is determined by specifications. Each requirement is either met or not met on a stated date, and your grade follows from which ones you complete. The quality of your app does not lower your grade. Failing to do the work does.

Layoffs are real. Failure to meet the requirements of this project can result in being laid off, which means a course grade of F. See the section Professionalism and Full-Stack Projects in the syllabus for the full range of conduct that can lead to a layoff. It extends well beyond this project.

4. Project Guidelines

- Your project must be a mobile application, built in Android Studio.
- Use the languages and tools Android Studio provides. Kotlin is recommended; Java is fine.
- Complete the capability map in your proposal, so you have thought about what goes into the app you are building.

- Do your own work, but take advantage of tools that exist. You may use development tools, generative AI, tutorials, and references, provided you document what you used and explain your role in the result.
- Creativity and customization count. Build something that feels like yours.
- All project materials must have a copy in your project repository, except videos hosted elsewhere. Organize the repository so a reader can find things.
- Keep every AI conversation from the start of the project. These are artifacts of your work and are submitted with your logs and final submission.
- If you want to pursue an idea that does not fit these guidelines, talk to me first.
- Ask me questions directly. Other instructors and advisors will not know the expectations for this project.

Taking Two of My Courses at Once

If you are taking two of my courses at the same time, you may build on the same idea in both. Everything else is separate. Each course gets its own work, its own logs, its own documentation, and its own presentation covering that course's portion. Some overlap in the underlying idea is expected and fine, but work submitted for one course does not count for the other. Two courses means two courses' worth of work, so plan accordingly. If this is your situation, come talk to me when you propose and we will sort out the specifics.

Continuing a Project From a Previous Course

Continuing a project from a previous course of mine is fine, and often a good idea. What matters is that you actually continue it. Where the project stood at the end of that course is your starting point, not your deliverable. Everything you are graded on here is what you build on top of it this semester. With your proposal, record that starting point: a repository tag, a commit hash, or a short note describing where the project stood when the previous course ended.

5. Mobile App Categories and Ideas

You are proposing your own app. What follows is a set of categories to help you figure out what kind of thing you want to build.

The examples under each category are illustrations of what apps in that category tend to look like. They are not a list to pick from, though there is nothing wrong with being inspired by one. Most apps fall into one of these categories, many span two or three, and a genuinely original idea may fit none of them. On your proposal you will name the categories your app falls into, or describe your own.

It Does Not Have to Be Original

Nobody expects you to invent an app that has never existed. Build a version of an app you have seen and liked, rebuild a tool you already use, or make your own take on something you enjoy. What makes it yours is the building and the choices you make along the way, not the novelty of the concept.

If Nothing Here Fits

If your idea does not fit any of these categories, describe it in your own words on the proposal and we will talk about it. Original ideas are welcome. It still has to be a mobile app.

Choosing and Sizing

Pick something you would be a little annoyed to abandon. Motivation is the resource that runs out first, and interest is what keeps a semester project alive in week ten.

A mobile app can balloon fast. If an idea feels too large, narrow it to a few screens done properly rather than a dozen half-built. If it feels too small, add a dimension such as saving data on the device, pulling data from the internet, or using a device feature like the camera or location. We will right-size it together when I review your proposal.

Table 1. *Mobile app categories and ideas*

A List of Categories and Ideas

Utility and Productivity

Apps that help someone get something done.

- To-do list or task manager with reminders
- Habit or streak tracker
- Budget or expense tracker
- Note-taking or journaling app
- Unit or currency converter with interface
- Countdown or event timer for something you care about

Information and Reference

Apps that organize or surface information.

- Recipe browser with search and favorites
- Study or flashcard app
- Local guide for restaurants, trails, or events
- Plant, animal, or product identifier
- Reference app for a domain you know well
- Personal catalog for books, media, or a collection

Health and Fitness

Apps that track or encourage a habit or activity.

- Workout logger with history
- Water, sleep, or step tracker
- Meditation or breathing timer
- Meal or calorie logger
- Stretching or exercise guide
- Mood tracker with trends over time

Device-Powered

Apps whose whole point is using what the phone can do.

- Camera app with a twist (scanning, overlays,

Games

Something to play. Simple is fine; polished-and-small beats ambitious-and-broken.

- Trivia or quiz game with categories
- Card or dice game
- Tile, match, or puzzle game
- Memory or reaction game
- Text-based adventure with choices
- Idle or clicker game with saved progress

Media and Creativity

Apps built around sound, images, or making things.

- Music or sound-effect player
- Photo gallery with filters or tagging
- Drawing or sketch pad
- Meme or image caption maker
- Simple video or audio recorder
- Color palette or design tool

Social and Community

Apps built around people, sharing, or groups. Keep the scope realistic; a full social network is too much, but a slice of one is not.

- Shared shopping or packing list
- Event or meetup planner for groups
- Local message board or bulletin
- Photo-sharing prototype for a group
- Poll or voting app for a group
- Study-group or team coordinator

filters)
- Location-based reminder or check-in app
- Compass, level, or measurement tool using sensors
- Step or motion detector
- Flashlight or utility toolkit
- Augmented-reality or map-based experience

6. What Goes Into a Mobile App: The Capability Map

With your proposal you complete a map of the capabilities your app will use. The purpose is not coverage. The purpose is to make you think about what actually goes into a mobile app before you start building it, and to notice the pieces your app will need.

This is not a checklist. You are not expected to use every capability. Nobody is counting. A short honest map is worth more than a padded one. Mark what your app will genuinely use and leave the rest alone.

The examples under each capability are prompts to think with, not requirements. Use the Other row for anything your app needs that is not listed.

Table 1. *Mobile app capabilities to consider*

Capability	What it means, with examples
Screens and Navigation	Multiple screens and moving between them. Example: a home screen, a detail screen, a settings screen, and the buttons or gestures that move between them.
User Input and Forms	Taking input from the user. Example: text fields, buttons, switches, date pickers, and checking that what they entered makes sense before you use it.
Displaying Lists of Data	Showing collections of items. Example: a scrolling list of tasks, a grid of photos, cards the user can tap to open.
Local Data Storage	Saving data on the device so it survives closing the app. Example: saved notes, a to-do list that is still there tomorrow, user settings that stick.
Networking and Outside Data	Getting data from the internet. Example: calling a public API for weather, sports scores, or recipes, and showing the results in your app.
Device Features	Using the hardware. Example: the camera, photo library, GPS and location, the microphone, or motion sensors.
Notifications and Alerts	Getting the user's attention. Example: a reminder notification, an alarm, an in-app alert when something needs a response.
Media	Sound, images, and video. Example: playing a sound effect, showing images, embedding a video, recording audio.
App State and Lifecycle	Handling how a mobile app actually runs. Example: remembering where the user was, surviving a screen rotation, saving progress when the app goes to the background.

Capability	What it means, with examples
Visual Design and Theming	Making it look intentional. Example: consistent colors and spacing, a light and dark theme, icons, a layout that works on different screen sizes.
Permissions	Asking the user for access. Example: requesting permission to use the camera, location, or notifications, and handling it gracefully if they say no.
Other	Anything your app needs that is not listed above, such as maps, authentication and login, in-app purchases, sharing to other apps, or a background service.

You do not have to know at proposal time exactly how each piece will work. A reasonable plan is enough, and the map can change as the app develops. You update it at the end with what you actually used.

7. Schedule

These dates are fixed.

Table 2. *Project schedule*

Week	What Is Due	Format
2	Proposal	Proposal with mapping
4	Log 1	Log
6	Log 2	Log
7	Progress video	5 to 7 minute demonstration, recorded or live
10	Log 3	Log
12	Log 4	Log
14	Log 5	Log
17	Final submission	Deliverable and repository, presentation, documentation form

See the Tentative Schedule in Course Information for specific due dates.

Logs use the log template provided with the project materials. Use the template; do not invent your own format.

8. What Progress Looks Like

Progress is not the same as finished features. On a project you designed yourself, progress is whatever moved you closer to being able to build the thing, and that varies enormously depending on what you ran into.

Progress includes

- Learning a tool, library, or part of Android you needed. If you spent a week getting the camera or a database working and understanding how it connects, that is progress. You

know something now that you did not know before.

- Trying an approach that failed and understanding why. A dead end you can explain is worth more than a feature you copied without understanding.
- Design work: sketching your screens, working out how the user moves through the app, deciding what data you need.
- Setting up infrastructure: the repository, the project skeleton, the emulator, the build.
- Reading and research: finding out how something is normally done before doing it.
- Getting help. If you came to me or to someone else with a problem and worked through it, that is progress and it belongs in your log.
- Writing code that works, obviously. But this is one form of progress among several, not the only one.

Progress does not include

- Nothing at all. "I have been busy" is not progress.
- Trivial work that did not move the project. Renaming files, reformatting code, or reorganizing folders is maintenance, not progress.
- Plans and promises. What you intend to do next month is not something you did.
- A statement that you made no progress. Reporting the absence of work is not a substitute for the work.
- Excuses and explanations for why nothing happened. These may be entirely true, and they still are not progress.
- Work you cannot describe. If you cannot say what you did and what came of it, there is nothing to report.

The honest test is whether you can point at something and say what it cost you and what you got out of it. If you can, that is progress, and it does not matter that it was not the thing you planned to do that week.

A log reporting no progress is a missed log. It counts exactly the same as not submitting one at all. Given how many things count as progress, and that simply coming to talk with me about a problem counts, there is no reason to end up here.

When It Is Not Going Well

Projects stall. Tools misbehave. That is not a problem in itself, and it is not something you will be penalized for.

What matters is what you do about it, and when. If you are stuck, deal with it then: come see me, send an email, ask for help. Do not wait for the next log to mention it, since the log is a report on what already happened, not a request for rescue.

If you need more time, spend a token. That is what tokens are for, and using one is the professional move, not a failure.

Going quiet is the one thing that does not work. A student who comes to me in week nine because the architecture is not working is a student I can help. A student who says nothing until week seventeen is a student I could not help, and by then the semester is gone.

Failing at something is fine. Not dealing with it is not.

9. The Progress Video

Due Week 7. Five to seven minutes. This is a technical demonstration of what you have built, not a status report and not a reflection.

Your project is not expected to be complete, and parts of it may not work yet. That is fine. What matters is that you can clearly explain what you built, what you attempted, and why you made the choices you made.

This is a chance to show what you accomplished, not to account for what you did not. The flexibility built into this project, the schedule you designed yourself and the tokens you hold, already gives you room to adjust scope and timeline throughout the semester. Because of that, there is no need to justify a lack of progress in this video.

The video must not include reflections, apologies, excuses, or explanations for why more was not accomplished. A video that is primarily apologies or excuses rather than a demonstration of completed work will be rejected and does not meet the specification. Frame it as an opportunity to show what you have built and take some pride in it.

Three ways to do it

- Record a video and share a link (OneDrive, YouTube, or your repository)
- Meet with me in person, scheduled in advance
- Meet with me by video conference, scheduled in advance

If you meet in person or by video conference, schedule it far enough ahead that it happens by the deadline. For a live session, what you submit is the date and time we met rather than a link.

Format

- Five to seven minutes, whichever way you do it
- Screen recording or screen sharing, so I can see the work
- Slides are optional. Demonstration is required. Production quality does not matter.

Suggested Structure

- **Reminder (30 seconds):** app name, what it does, who it is for
- **What you have built (3 to 4 minutes):** show it. The app running, or the screens, code, and pieces that exist. If it runs on an emulator or device, show that. If it does not run yet, walk through the code. This is the part I most want to see.
- **What you taught yourself (1 minute):** show a tool, part of Android, or technique you did not know before. Demonstrate it rather than narrating what the experience taught you.
- **What is next (1 minute):** features in progress, immediate next steps, known problems
- Seven minutes is part of the exercise. Your manager has a few minutes and wants to see what you have. Decide in advance what is worth showing rather than narrating everything you did.

10. The Ten Specifications

Table 3. *The ten specifications*

#	Specification	Due	Type
1	Proposal submitted and approved	Week 2	Gate
2	Log 1 submitted, showing progress	Week 4	Ladder
3	Log 2 submitted, showing progress	Week 6	Ladder
4	Progress video submitted, demonstrating real material	Week 7	Ladder
5	Log 3 submitted, showing progress	Week 10	Ladder
6	Log 4 submitted, showing progress	Week 12	Ladder
7	Log 5 submitted, showing progress	Week 14	Ladder
8	Final deliverable and complete repository submitted	Week 17	Gate
9	Final presentation given, including demonstration	Week 17	Gate
10	Final documentation form submitted	Week 17	Gate

11. How Your Project Grade Is Determined

Two things are gates. Everything else sets the grade.

Gate 1: The Proposal

No approved proposal means you are laid off, and there is no project to grade, which is an F.

Gate 2: The Week 17 Submission

All three Week 17 items are required: the final deliverable and repository, the presentation, and the documentation form. Missing any one of them is an F, regardless of everything else you did.

The Grade Ladder

If both gates are cleared, your grade is set by the progress video and your logs. A log counts as met if it was submitted on time and reports real progress, or if it was covered by a token.

Table 4. *Grade ladder*

Grade	Points	Requirements
A	100	Progress video submitted, and all 5 logs met.
B	85	Progress video submitted, and 4 of 5 logs met.
C	75	Progress video submitted, and 3 or fewer logs met.
D	60	Progress video not submitted.
F	0	Any gate not met (no approved proposal, or any Week 17 item missing).

These are the only possible scores for this project. There is no partial credit within a grade. You either met the specifications for that grade or you did not.

The progress video is required for a C or higher. Without it, no number of logs will move you

above a D. You have two tokens, a full week of recovery time, and three different ways to deliver it, so there is no real reason to miss it.

Note what else this means. A log you handled with a token still counts as met. Communicating and adjusting costs you a token, not a grade. Going silent is the thing that moves you down the ladder.

What Counts as Met

- A log is met when it is submitted on time and reports real progress. Length does not matter; there is no page requirement. A log reporting no progress does not count. Use the log template.
- The progress video is met when it demonstrates actual project material. A video that only describes plans does not meet it.
- The final deliverable is met when your code and complete repository are submitted and correspond to the app you proposed.
- The presentation is met when you give it and it includes a demonstration. You are not graded on speaking skill.
- The documentation form is met when it is submitted with its fields completed.

A submission missing the parts a requirement calls for has not been made, and the specification remains unmet. This is a check that the required components are present, not a judgment of how good they are.

12. Tokens

You receive two tokens at the start of the semester. Things come up that nobody plans for, and tokens exist to cover them. Using one is the professional move. It is what you do instead of going quiet.

How tokens work

- Two tokens for the entire semester, and no additional tokens are granted.
- A token may be used on the proposal, any log, or the progress video. It is a shared pool, so what you spend early is not available later.
- Tokens cannot be applied to any Week 17 item. The term ends and there is no time to accommodate them.
- A token buys either an extension or one revision and resubmission of that item.
- A log covered by a token counts as met for the grade ladder.

Timing, which is the part that matters

- You must request a token within three days of the original deadline.
- A token moves that item's deadline to seven days after its original due date. This date is fixed.
- Requesting sooner gives you more time to do the work. Requesting later does not move the date; it only leaves you less time.
- A token applies only to the current item. Once the recovery period ends, that item is closed permanently and cannot be reopened later with a token.

Read that again. If you miss a Monday deadline and come to me that same day, you have until

Sunday. If you wait until Wednesday, you still have until Sunday. Delay costs you working time, not deadline time.

13. Final Submission and Presentation

Three things are due in Week 17: your deliverable and repository, your presentation, and your documentation form. All three are required. Missing any one of them is an F, regardless of everything else you did.

On timing. Tokens cannot be applied to any Week 17 item. The term ends and there is no time to accommodate an extension. Plan accordingly.

Deliverable and Repository

Everything goes in the repository. All of it.

- All source code and the full Android Studio project
- Any databases, assets, and resource files
- All documents, sketches, notes, and design work
- All AI conversations, prompts and outputs, as PDFs
- Anything else you produced for this project

Work that does not run is still submitted. Broken code, abandoned attempts, half-finished screens, and dead ends all go in. They are evidence of what you did, and this project has never required that everything work. Hiding them serves no purpose.

Organize the repository so a reader can find things, and include a README that says what the app is, what works, and how to build or run it.

Your app does not have to be complete. It has to be the app you proposed, and it has to be real. An app that fell short of what you hoped still meets this specification. Account for what happened on your documentation form.

Presentation

Five to ten minutes, live or recorded. This is about the app, not about you.

No reflection in the presentation. Do not narrate what you learned about yourself, how you grew, or what the experience taught you. That goes on the documentation form. This is a demonstration of what you built. Showing a tool or technique you learned is welcome, because that is showing the work.

Requirements

- A demonstration is required, live or recorded. Slides are optional.
- Show the app running on an emulator or device if you can. If it does not run, walk through your code and the pieces you built.
- If you present live, test everything first and have a backup recording or screenshots.

Suggested Structure

- **Introduction (1 minute):** who you are, app name, what it does
- **Motivation (1 minute):** what you wanted to build and why you chose it
- **Demonstration (4 to 7 minutes):** the app running or the code walkthrough, key screens

and features, important choices, interesting challenges, tools or techniques you picked up

- **Closing (30 seconds):** what you would add next

You are not graded on being a polished speaker. Speak clearly, keep the demo visible, and do not read from a script. The specification is that you gave the presentation and demonstrated the work.

Documentation Form

Fill out the provided form. It is a form, not a report. There is no length requirement and no essay. A few lines per field is all that is wanted, and it should take fifteen or twenty minutes if you actually did the work.

Its fields cover your app name and description, final status, the updated capability map, a few lines on what you learned and what you would do differently, resources used, AI use, and your links.

14. AI and Outside Resources

This project operates under different terms than the weekly assignments in this course. For the project, you may use generative AI and other outside resources, provided you document what you used and what your role in the result was.

Keep every AI conversation from the start of your project. These are artifacts of your work and must be submitted with your logs and in your final submission as PDFs. You are not graded on the content of these conversations. I want to see how you worked through problems, and having a record of how AI was actually used in this course matters beyond this class.

Be honest about it. There is no reason to claim you used AI only to learn something if it generated your starting code. Documented use is expected and is not penalized. Undisclosed use is a different matter and falls under the academic integrity policy in the syllabus.